

SKALA

4) AI 서비스의 고도화 생성형 AI 서비스 개발

4. Vector DB

2026.10

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

오늘 우리가 만들 것

교육 목표

- ▶ 임베딩·칭킹·검색·재순위화로 이어지는 RAG 파이프라인의 원리를 '비유'로 이해한다
- ▶ FAISS → Qdrant, Hybrid Search, Re-ranking을 직접 손으로 구축해본다
- ▶ Vector DB의 개념을 서로간에 Talking About 한다(오늘의 핵심!)
- ▶ 내 도메인 문서로 RAG 시스템을 설계할 수 있는 기준을 갖는다

선행 지식: 기초 RAG 개념(문서→Embed→검색→응답) 정도면 충분합니다

이 과정을 대하는 방법 — 세 가지 원칙

직접 손으로

코드를 눈으로만 보지 말고
반드시 직접 입력해봅니다.

에러도 훌륭한 학습입니다.

내 언어로 설명하기

전공 용어가 어려우면
나만의 비유로 바꿔 말해보세요.

옆 사람에게 설명할 수 있으면
진짜 이해한 것입니다.

함께 이야기하기

막히는 부분은 옆 사람과
먼저 상의하세요.

실습 전 'Takling About' 시간에
팀 단위로 정리합니다.

이론보다 '체감'이 목표 — 막히면 언제든지 질문하세요

Vector DB

1. Orientation
2. Embedding Model
3. Chunking/Vector DB
4. Hybrid Search & Re-ranking
5. Agentic RAG
6. 최신 현황
7. Talking About...
8. 실습

4) AI 서비스의 고도화 생성형 AI 서비스 개발 – Vector DB

1. Orientation

RAG 복습 — 기초에서 오늘로

기초 과정에서 배운 것

- **RAG 파이프라인**
: 문서 → **Embed** → 검색 → **LLM** 응답
- **임베딩**: 문장을 벡터로 바꾸는 개념
- **Vector DB**
: 유사도 검색 **DB** 개념 (**FAISS** 기초)
- **Chunking**
: 정해진 크기로 자르기 (**Fixed Chunk**)
- 검색 품질: **Top-K** 결과를 그대로 사용

오늘 심화할 내용

- 다국어 지원, 상황별 모델 선택 기준 (**BGE-M3** 등)
- **Qdrant** 운영 — **HNSW** 튜닝, 필터 검색
- 의미 단위로 자르기, **Parent-Child** 전략
- **Dense** + 키워드를 합치는 **Hybrid Search**
- **Re-ranking**으로 순위를 한 번 더 다듬기

이론보다 '체감'이 목표 — 막히면 언제든지 질문하세요

왜 더? — 기초 RAG의 한계

예시

책상 위에 딱 5권만 올려두고 시험을 보는 상황

- 기초 RAG는 질문이 오면 관련 문서 Top-5만 골라 LLM에게 건네줍니다.
- 그런데 책이 두꺼우면(① Context Window 제약) 중요한 페이지가 통째로 빠지고,
- 엉뚱한 책이 섞이면(② Retrieval Noise) 헛갈려서 오답을 내고,
- 여러 책을 넘나들며 답해야 하는 질문(③ Cross-Document 추론)엔 손을 못 대고,
- 책에 없는 '고유한 단어 하나'(④ 키워드 의존)를 놓치면 검색 자체가 실패합니다.

오늘 이후 할 수 있게 되는 것들

오늘 이후
할 수 있는
것들

- **BGE-M3** 등 최신 임베딩 모델을 선택·적용 할 수 있다.
- **Semantic Chunking**과 **Parent-Child** 전략을 실무에 적용 할 수 있다
- **BM25 + Dense Search Hybrid** 파이프라인을 직접 구축 할 수 있다
- **BGE-Reranker**로 검색 품질을 추가 개선할 수 있다
- **Agentic RAG** 아키텍처를 설명하고 설계할 수 있다
- **MEMO** 논문의 핵심 아이디어를 이해하고 기존 **RAG**와 비교할 수 있다
- **Qdrant** 기반 **Production RAG** 시스템을 구성할 수 있다

Embedding Model — 검색의 첫 단추, 모델 고르는 눈을 기릅니다

4) AI 서비스의 고도화 생성형 AI 서비스 개발 – Vector DB

2. Embedding Model

- Embedding 심화
- 검색 품질 고도화

임베딩이란? — 먼저 감을 잡아봅시다

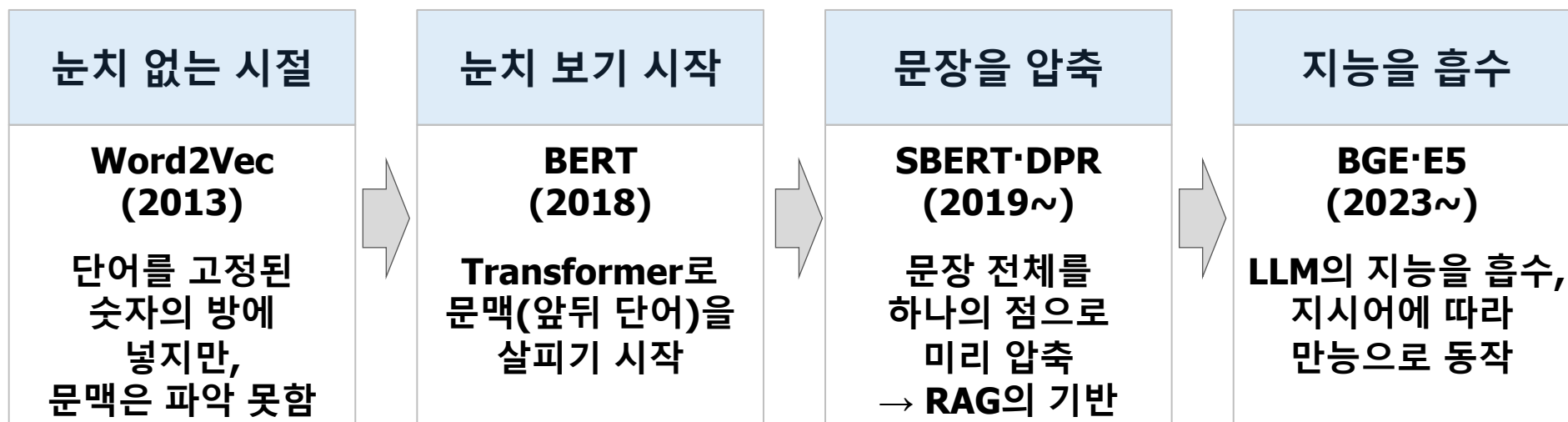
예시

단어의 '의미'를 지도 위의 좌표로 바꾸는 것

- '강아지'와 '고양이'는 의미가 비슷하니 지도 위에서 가까운 곳에 찍히고,
- '강아지'와 '자동차'는 의미가 다르니 멀리 떨어진 곳에 찍힙니다.
- 이 좌표(숫자 목록, 예: $[0.2, -0.5, 0.8, \dots]$)가 바로 '임베딩 벡터'입니다.

검색이란 결국 '내 질문의 좌표와 가장 가까운 좌표를 찾는 것'입니다.

임베딩 모델의 진화 — 4단계 이야기



핵심 흐름: 단어 단위 → 문맥 이해 → 문장 압축(RAG 대중화) → LLM급 만능 모델

모델 계보 한눈에 — Word2Vec에서 BGE-M3까지

모델	한 줄 정리	실무
Word2Vec (2013)	단어 단위 고정 벡터, 문맥 무시	경량 키워드 추천, 레거시 유지
BERT (2018)	양 방향 문맥 이해, 문장 임베딩엔 부적합	분류, 파인 튜닝용
Sentence-BERT (2019)	문장을 하나의 벡터로, 영어 중심	영어 문서 빠른 유사도 검색
OpenAI text-embedding-3	API기반, 3072차원, 고품질	빠른 프로토타이핑, 다국어
BGE-M3 (2024)	다국어, Dense + Sparse + CoBERT 통합	한국어 우수
E5-mistral-7b (2024)	LLM 기반, MTEB 최상위, 무거움	오프라인 배치 임베딩

오늘 실습은 BGE-M3 사용 — 무료, 한국어 우수, 실무에서 가장 많이 쓰는 선택

[참고] Massive Text Embedding Benchmark

임베딩 모델들의 올림픽 순위표로 56개 데이터셋, 8가지 과제(검색·분류·클러스터링 등)로 모델 성능 종합 평가

항목	BERT계열	E5-mistral-7b
파라미터	110M	7,110M (65배)
GPU 메모리	2GB	16GB 이상
속도	빠름	느림
실시간 서비스	가능	어려움

**“F1 레이싱카처럼 빠르고 강하지만, 일반 도로에서 출퇴근용으로 쓰기엔 무리”
(E5-mistral-7b)**

유사도(거리)는 어떻게 잴까? — 4가지 방법 비유

방법	비유로 이해	언제?
Cosine Similarity	나침반 방향이 같은가? (크기는 안 봄)	대부분의 임베딩 검색 (기본값)
Dot Product	방향 + 크기를 같이 볼 때	이미 정규화된 벡터, 속도 중시
Euclidean (L2)	두 점 사이 직선 거리 (자로 재기)	절대적인 거리가 의미 있을 때
Manhattan (L1)	바둑판 도로에서 길 찾기 (대각선 금지!)	이상치에 강해야 할 때

실무에서는 대부분 **Cosine Similarity** 사용

수백만 개 벡터, 어떻게 빠르게 찾을까? — ANN

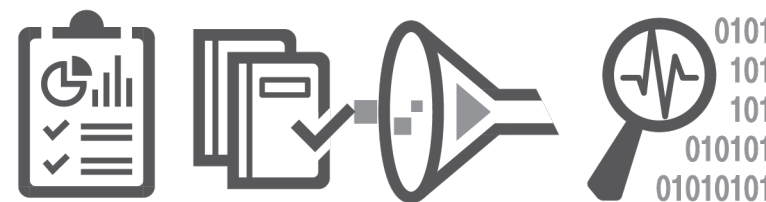
Brute Force (완전 탐색)

- 100만 개를 전부 하나씩 비교
- 정확도: 100%
- 속도: 10초 소요
→ 실시간 서비스 불가능



ANN (근사 탐색)

- 99% 확률로 1등만 빠르게 탐색
- 정확도 : 95~99%
- 속도 : 0.1초 소요
→ 실시간 서비스 가능



정확도를 살짝 희생하고 속도를 극적으로 향상 — 사용자는 차이를 못 느낍니다

HNSW — 실무 표준 ANN, '고속도로' 비유

Layer 2 — 고속도로 

큰 도시(대표 벡터)만 연결해 멀리 빠르게 이동

Layer 1 — 국도 

중간 크기 지점들을 연결해 범위를 좁힘

Layer 0 — 골목길 

모든 벡터를 촘촘히 연결해 정확하게 도착

핵심 파라미터: $M(\text{연결 수}) \cdot ef_construction(\text{구성 정확도}) \cdot ef_search(\text{검색 시 정확도})$

검색 품질, 숫자로 확인하기

Recall@K

K개 결과 중
정답이 있는 비율

RAG에서
가장 많이 씀

Precision@K

K개 결과 중
정답의 비율

검색 정밀도 측정

MRR

첫 정답이
몇 번째에 나왔는지
의 평균

nDCG@K

순위가 높을수록
가중치를 더 주는
종합지표

오늘 실습에서는 Recall@5를 기준으로 청크 크기·검색 방식을 비교합니다

실무 임베딩 모델 선택 가이드

이렇게
고르세요

- 처음 시작한다면
→ **BGE-M3** (무료, 다국어·한국어 우수, 실무 최우선 권장)
- **API**로 빠르게 붙이고 싶다면
→ **OpenAI text-embedding-3** (유료, 편리)
- 최고 성능이 꼭 필요하다면
→ **E5-mistral-7b** (단, GPU 메모리 **16GB**↑ 필요)
- 보안이 중요한 **On-premise** 환경이라면
→ **BGE-M3** 로컬 구동

MTEB 1위 ≠ 내 서비스 1위 — 속도·비용·인프라를 함께 고려해서 선택합니다

핵심 요약

임베딩 : 의미를 좌표(벡터로)

모델 선택 : 실무에 적합한 걸로

유사도 : Cosine 이 기본

ANN/HNSW : 속도와 정확도의 균형

품질지표 : Recall@K로 확인

Chunking & Vector DB
잘 자르고 잘 저장하기

4) AI 서비스의 고도화 생성형 AI 서비스 개발 – Vector DB

3. Chunking/Vector DB

- **Chunking Engineering** : 어떻게 자르느냐가 검색 품질을 결정한다
- **RAG 설계 패턴**

왜 Chunking이 필요한가?

Chunking을 통한 효율성 극대화 – LLM에 책 전체를 넣을 수 없다

문제

- 100페이지 PDF
→ LLM에 통째로 입력?



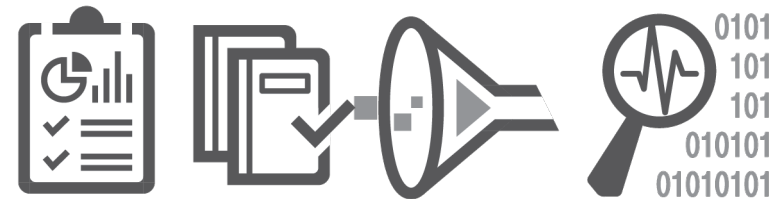
✗ 토큰 한도 초과

✗ 비용 폭발

✗ 응답까지 수 분 소요

해결

- PDF → 200개 청크로 분할
→ Vector DB에 저장
- 질문에 관련된 것만 검색
→ Top-5 청크만 LLM에 전달



핵심: 필요한 부분만 똑똑하게 골라 전달한다

그럼 얼마나 자를까?

크게? 작게?

작은 청크 (100~300 토큰)

- 장점
 - 정밀한 벡터 표현
 - 노이즈 감소
- 단점
 - 맥락 단절
 - 재조합 필요



큰 청크 (700~1000 토큰)

- 장점
 - 풍부한 맥락
 - 답변 품질 좋음
- 단점
 - 임베딩 품질 저하
 - 노이즈 증가



🔥 실무 기본값: 300~500 토큰 + 10% 오버랩 🔥

overlap 필요 이유

문장, 문맥에 대한 의미 단절 회피

【 Overlap이 없을 때 】

청크 1: "...AI 기술이 발전하면서"

청크 2: "새로운 가능성이 열렸다..."



【 Overlap이 있을 때 】

청크 1: "...발전하면서 새로운"

청크 2: "발전하면서 새로운 가능성..."



 권장: `chunk_size`의 10~20% | 예) 500토큰 → 50토큰 오버랩 

4가지 Chunking 전략을 한 눈에

전략	한 줄 설명	속도/품질	추천
Fixed Size	정해진 길이로 무조건 자르기	빠름 / 보통	빠른 프로토타이핑
Recursive	문단→줄바꿈→문장 순으로 자연스럽게	빠름 / 좋음	대부분의 문서 (기본 권장)
Semantic	의미가 바뀌는 지점에서 자르기	느림 / 최고	학술 논문, 전문 기술 문서
✓ Parent-Child	작게 검색 + 크게 전달	보통 / 매우 좋음	실무에서 권장

처음 시작 → Recursive | 실무 배포 → Parent-Child | 최고 품질 → Semantic

전략 ① Fixed Size

가장 단순한 방법

장점

- ✓ 구현 빠르고 단순
- ✓ 예측 가능한 크기
- ✓ 모든 문서 타입 적용

단점

- ✓ 문장 중간 잘릴 수 있음
- ✓ 의미 단위 무시

【 코드 예시 】

```
splitter =  
RecursiveCharacterTextSplitter(  
    chunk_size=500,  
    chunk_overlap=50,  
    separators=['\n\n', '\n', '.', ', ']  
)
```

구조 없는 텍스트, 빠른 프로토타이핑

전략 ② Recursive

자연스러운 경계 찾기

장점

- ✓ 문단 경계 존중
- ✓ 자연스러운 분할
- ✓ LangChain 기본값

단점

- ✓ 구분자 설정 복잡

【 코드 예시 】

```
# 구분자 우선순위
separators=[
    '\n\n', # 문단
    '\n',   # 줄바꿈
    '!',    # 문장
    ' ',    # 단어
]
```

대부분의 일반 문서(기본 권장)

전략 ③ Semantic

의미가 바뀌는 곳에서 자르기

장점

- ✓ 의미 단위로 분할
- ✓ 가장 자연스러운 청크

단점

- ✓ 임베딩 연산 필요 → 느림
- ✓ 비용 발생 (API 사용 시)

【 코드 예시 】

```
from langchain_experimental
    .text_splitter import
    SemanticChunker

splitter = SemanticChunker(
    embeddings_model
)
```

학술 논문, 전문 기술 문서

전략 ④ Parent-Child

실무 최우선 권장

장점

- ✓ 검색 정밀도 ↑ (Child)
- ✓ 맥락 풍부 ↑ (Parent)
- ✓ 두 장점을 동시에

단점

- ✓ 저장 공간 약 2배
- ✓ 구현 복잡도 있음

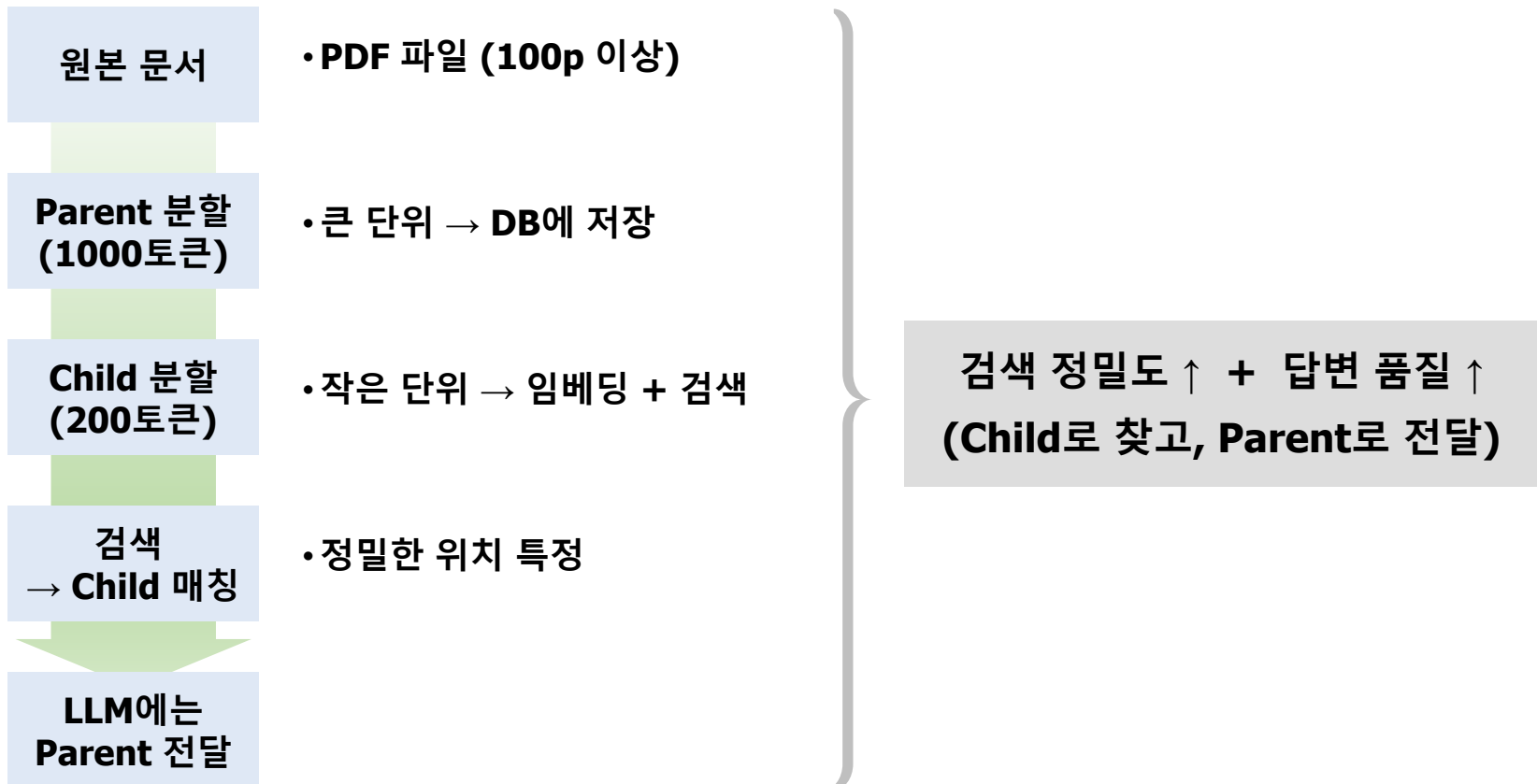
【 코드 예시 】

```
retriever = ParentDocumentRetriever(  
    vectorstore=vectordb,  
    docstore=InMemoryStore(),  
    child_splitter=child_sp,  
    parent_splitter=parent_sp  
)
```

실제 실무 환경

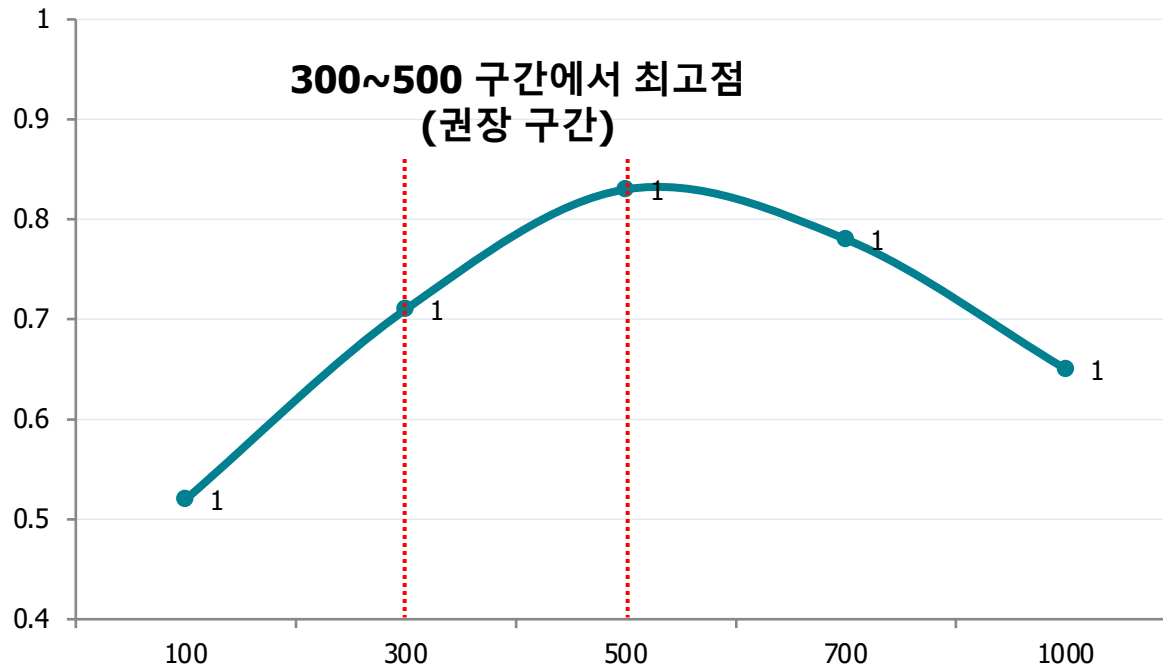
Parent-Child 상세

【 왜 강력한가? 】



Chunk 크기가 검색 품질에 미치는 영향

청크 크기별 Recall@5 (예시)



문서 유형	권장 Chunk 크기
짧은 FAQ	100 ~ 200 토큰
일반 보고서	300 ~ 500 토큰
법률 계약서	500 ~ 800 토큰
기술 매뉴얼	400 ~ 600 토큰

Chunk 크기가 검색 품질에 미치는 영향(예시)

예시

▪ Recall@5

- 상위 5개 검색 결과 안에 정답이 있는가?
(100개 질문에 평균낸 값)
- 1.0 : 모든 질문에 상위 5개 안 정답 존재
- 0.5 : 절반만 정답

▪ 100 토큰(0.52)이 낮은가?

- Chunk가 낮으면 맥락이 단절
(HNSW는 계층형 구조로..
→ HNSW 계층형 / 구조로...)

▪ 1000 토큰(0.65)이 낮은가?

- Chunk가 크면 노이즈 증가
(어떤 질문에도 애매하게 관련 있어
보여서 오히려 검색이 부정확해짐)

【 왜 300~500토큰(0.83)이 최고점인가? 】

충분한 맥락 + 적당한 집중도
균형점

한 개념을 설명하기에 충분한 분량

불필요한 내용이 섞이지 않음

임베딩 벡터가 하나의 명확한
의미를 표현

Chunk 크기가 검색 품질에 미치는 영향(예시)

문서 유형	권장 Chunk 크기	이유
짧은 FAQ	100 ~ 200 토큰	- 질문-답변 하나가 원래 짧음. - 크게 자르면 여러 Q&A가 섞임
일반 보고서	300 ~ 500 토큰	- 단락 하나가 하나의 개념. - 기본값으로 가장 무난
법률 계약서	500 ~ 800 토큰	조항 하나가 길고, 앞뒤 문맥이 법적 의미에 영향을 줌
기술 매뉴얼	400 ~ 600 토큰	기능 설명 + 예시 코드가 함께 있어야 완전한 정보

메타데이터 – Chunk에 다는 꼬리표

도서관 책 태그

- ✓ 책 자체 = 청크 텍스트
- ✓ 어느 선반? = 카테고리
- ✓ 몇 번째 책? = **doc_id**
- ✓ 언제 입고? = **date**
- ✓ 누가 썼나? = **author**

【 코드 예시 】

```
# 메타데이터(Payload) 예시
{
  "doc_id": "annual_2024",
  "page": 23,
  "category": "재무",
  "department": "경영기획",
  "date": "2024-03-15",
  "language": "ko"
}
```

필터 활용 예시

'재무팀 2024년 문서에서만 검색'

'3페이지 이후 내용에서만 검색'

메타데이터 필터 검색 - 코드로 보기

【 코드 예시 】

```
# 일반 검색
results = client.search(
    collection_name="rag_docs",
    query_vector=q_vec,
    limit=5
)

# 필터 검색 (5페이지 이후만)
results = client.search(
    collection_name="rag_docs",
    query_vector=q_vec,
    query_filter=Filter(must=[
        FieldCondition(
            key='page',
            range=Range(gte=5)
        )
    ]),
    limit=5
)
```

언제 필터를 쓰나?

- ✓ 최신 문서만 검색 : **date** >= '2024-01-01'
- ✓ 특정 카테고리만 : **category** = '법무'
- ✓ 특정 페이지 범위 : **page** >= 10 AND **page** <= 50
- ✓ 특정 언어만 : **language** = 'ko'

Vector DB – 임베딩을 저장하고 검색하는 전용 DB

	일반 DB (Relational DB)	Vector DB
저장 대상	정형 데이터	임베딩 벡터
검색 방식	SQL (정확 매칭)	유사도 검색
인덱스	B-Tree	HNSW / IVF
쿼리	WHERE 조건	벡터 거리
결과	정확한 행	가장 유사한 것

FAISS Facebook,
라이브러리

Qdrant 오픈소스, 서버

Pinecone 클라우드 관리형

Chroma 경량, 로컬

FAISS vs Qdrant – 오늘 실습에 쓸 2가지

	FAISS	Qdrant
개발	• Facebook AI	• Qdrant Labs
방식	• 라이브러리	• 독립 서버 (Docker)
메타데이터	• 지원 안됨	• Payload 지원
필터 검색	• 불가	• 가능
Hybrid 검색	• 별도 구현	• 기본 지원
적합 상황	• 빠른 실험	• 실무 프로덕션

 실습: FAISS로 먼저 만들어보고 → Qdrant로 전환하며 차이를 직접 체감 

지금까지 배운 것을 연결하면 – RAG 전체 파이프라인



① 문서 로딩

PDF/Word
→ 텍스트 추출



② 청크+메타데이터

Recursive/Parent-Child



③ 임베딩

BGE-M3로 벡터 변환



④ **Vector DB** 저장

Qdrant에 저장



⑤ 유사도 검색

HNSW로 빠르게 탐색



⑥ **Top-K** + 필터

메타데이터로 좁히기



⑦ **LLM** 프롬프트

검색 결과 + 질문 전달



⑧ 최종 답변

사용자에게 응답

핵심 요약

Chunking 중요성 : 검색 정확도를 좌우

기본값 : 300~500 토큰 + 10% 오버랩

실무 : Parent-Child 전략

메타 데이터 : 카테고리, 날짜 등 태그

FAISS→Qdrant : 필터 필요하면 Qdrant

Hybrid Search & Re-ranking
검색을 더 정확하게

4) AI 서비스의 고도화 생성형 AI 서비스 개발 – Vector DB

4. Hybrid Search & Re-ranking

- Dense의 맹점을 BM25로 보완하고, Reranker로 정밀화

Semantic Search만으로는 부족하다



검색어: "PostgreSQL 17.2 설치 오류 코드 42P01"

Dense 검색 결과

- 데이터베이스 기초 개념 가이드
- SQL 쿼리 최적화 방법
- 데이터베이스 성능 튜닝
- NoSQL vs RDB 비교

필요한 결과 (없음!)

- PG 17.2 릴리즈 노트
- 42P01 오류 해결 가이드
- PG 설치 오류 모음

검색어: "PostgreSQL 17.2 설치 오류 코드 42P01" — 버전·에러코드를 '의미적으로' 처리 못함

Dense Search가 실패하는 6가지 유형

이런 도메인일수록 Hybrid Search 효과 극대화!

버전 번호

- Python 3.11.2
- Ubuntu 22.04 LTS

법령 번호

- 개인정보보호법 제17조
- ISO 27001:2022

에러 코드

- NullPointerException
- 42P01 오류

코드/수식

- $O(n \log n)$
- `SELECT * FROM table`

고유명사

- GPT-4o-mini
- Llama 3.1 405B

새로운 정보

- 학습 후 등장한
- 신제품명·신조어

BM25 — 40년간 검증된 최강 키워드 검색

예시

희귀하고, 많이 나올수록 → 관련성이 높다

- 조건 1: 찾는 단어가 문서에 많이 나올수록 점수↑ (PostgreSQL 5번 > 1번)
- 조건 2: 그 단어가 전체 문서에서 희귀할수록 점수↑ ('있다','하는'은 낮은 가중치)
- 1994년 발표, 학습 불필요(통계 기반), 새로운 단어도 즉시 처리 가능합니다.

Dense vs BM25 — 서로 다른 강점

항목	Dense (의미 검색)	BM25 (키워드 검색)
잘 하는 것	의미 이해, 동의어 처리	정확한 키워드 매칭
못 하는 것	버전번호, 에러코드	동의어, 문맥 이해
학습 필요	필요(임베딩 모델)	불필요(통계 기반)
새 단어 처리	어려움	즉시 가능

두 방법의 약점이 서로 보완됨 → 합치면 더 강해진다!

Dense vs BM25 — 서로 다른 강점

두 방법의 약점이 서로 보완됨 → 합치면 더 강해진다!

항목	Dense (의미 검색)	BM25 (키워드 검색)
검색 원리	벡터 유사도	단어 빈도 통계
잘하는 것	의미 이해, 동의어	정확한 키워드 매칭
못하는 것	버전번호, 에러코드	동의어, 문맥 이해
학습 필요	필요 (임베딩 모델)	불필요 (통계)
새 단어 처리	어려움	즉시 가능
다국어	모델에 따라 다름	토큰나이저 필요

Hybrid Search — Dense + BM25 동시에



검색어: "PostgreSQL 17.2 설치 오류 코드 42P01"

Dense 검색 (의미)

- Top-20 결과
- 의미 기반 → 관련 개념 찾음

BM25 검색 (키워드)

- Top-20 결과
- 키워드 기반 → 정확한 버전·코드 찾음

RRF 결합 → 최종 Top-10

1위. 42P01 해결 가이드

2위. PG 17.2 릴리즈 노트

Hybrid Search — 두 순위를 어떻게 합칠까? (RRF)

비유: 두 심사위원 점수 합산

심사위원 A (Dense)

- 1위: 문서X 2위: 문서Y 3위: 문서Z

심사위원 B (BM25)

- 1위: 문서Z 2위: 문서X 3위: 문서Y

단순 합산 X (점수 스케일 달라서 문제!)

```
# RRF 공식 (k=60 표준 상수)
score = 1/(60+rank_A) + 1/(60+rank_B)
# 점수 아닌 순위만 사용 → 안전한 결합
```

【 RRF 계산 결과 】

문서X : Dense 1위 + BM25 2위 | 0.0328

문서Z : Dense 3위 + BM25 1위 | 0.0327

문서Y : Dense 2위 + BM25 3위 | 0.0319

k=60은 표준 상수값 — 거의 바꿀 필요 없음

Hybrid Search 적용 전후 비교



검색어: "PostgreSQL 17.2 설치 오류 코드 42P01"



Dense만

- DB 설치 개요
- SQL 기초 문법
- DB 성능 최적화
- NoSQL vs SQL
- 인덱스 설계 가이드



Hybrid (RRF)

- PG 17.2 릴리즈 노트
- 42P01 해결 가이드
- PG 설치 오류 모음
- 버전별 설치 차이점
- 설정 파일 가이드

법률 | IT 기술문서 | 의료 | 금융 규정 | 제품 매뉴얼 → Hybrid가 빛나는 도메인

[참고] α (알파) 가중치 — Dense와 BM25의 비율 조절

← BM25 강화

균형 (기본값)

Dense 강화 →

BM25 100%

 $\alpha = 0.5$ (기본)

Dense 100%

BM25 강화
($\alpha=0.2\sim0.4$)

• 키워드 도메인

- 법률 조문, 에러 코드
- 제품 모델명, 규정 번호

균형
($\alpha=0.5$)

• 일반 RAG 서비스

- 처음 시작할 때 기본값
- 대부분의 상황에서 무난

Dense 강화
($\alpha=0.7$)

• 의미 검색 도메인

- 감성 분석, 주제 검색
- 고객 문의, 일반 대화

권장: $\alpha=0.5\sim0.7$ (Dense 우선) → 도메인 데이터 A/B 테스트 후 결정

Re-ranking – 한 번 더 정밀하게

예시

서류 전형(Hybrid) 다음, 대면 면접(Re-ranking)

- Hybrid 검색 = 서류 전형: 빠르게 50명을 추림 (근사적, 빠름)
- Re-ranking = 대면 면접: 질문과 문서를 '한 쌍씩 직접' 비교해 10명 선발 (느리지만 정밀)
- 전체 DB가 아니라 Top-50개만 다시 보기 때문에 느려도 실용적입니다.

Re-ranking – 한 번 더 정밀하게

Hybrid 검색 후에도 순위가 완벽하지 않다!

【 상황 】

- Hybrid 검색으로 Top-20 추출 완료
- 하지만 1위 문서가 실제로 가장 관련 없음
— 왜 그럴까?

→ Hybrid = 빠른 근사 방법

→ 문서와 쿼리를 '개별적으로' 비교

→ 더 정확한 방법 필요



면접 비교 : 서류 전형

- Hybrid 검색
- 빠르게 50명 추림 | 근사적, 빠름



면접 비교 : 대면 면접

- Re-ranking
- 직접 만나 10명 선발 | 정밀하지만 느림

Re-ranking 5단계 파이프라인

Step 1

사용자 질문 입력

Step 2

Hybrid Search → Top-50 추출 (빠른 근사 검색 | ~50ms)

Step 3

Reranker 입력 (질문, 청크) 쌍 50개 → 관련도 점수 | ~200ms

Step 4

점수 기준 재정렬 (0.97, 0.94, 0.82 ... 순서로 정렬)

Step 5

상위 5~10개만 LLM에 전달 (노이즈 제거된 고품질 컨텍스트 → 답변 생성)

Hybrid(~50ms) → Reranker(~200ms) → LLM → 최종 답변 (~1~3초)

우리 서비스에 Hybrid Search가 필요할까?

예시

이런 경우라면 필요합니다

- 검색어에 버전번호·에러코드·모델명이 자주 포함된다
- 법률·의료·금융 규정처럼 정확한 용어가 중요한 문서를 다룬다
- Dense만으로 Recall@5가 0.7 미만이다
- 사용자가 '정확한 내용이 안 나와요'라고 자주 불만을 제기한다

의심스러우면 Hybrid부터 적용하세요 — 항상 더 나빠지는 경우는 거의 없습니다

핵심 요약

Dense의 한계 : 버전, 코드, 고유명사 실패

BM25 : 40년 검증된 키워드 검색

Hybrid : RRF로 순위 안전 결합

Re-ranking
: Top-50 → Cross-encoder → Top-10

적용 기준 : 의심되면 Hybrid부터

***Agentic RAG & 최신 현황
한 단계 더 똑똑한 RAG***

4) AI 서비스의 고도화 생성형 AI 서비스 개발 – Vector DB

5. Agentic RAG

- 단순 검색을 넘어 추론하는 AI 시스템

이런 질문은 기존 RAG가 답 못합니다

"2023년과 2024년 매출을 비교해서 증감 원인을 분석해줘"

"이 논문 저자의 최근 3개 논문 핵심 내용을 요약해줘"

"경쟁사 A, B, C의 제품 가격과 특징을 비교한 표를 만들어줘"

공통점: 검색 1회로는 부족 — 여러 번 검색하고 스스로 판단해야 합니다

일반 RAG vs Agentic RAG – 무엇이 다른가?

항목	일반 RAG	Agentic RAG
검색 횟수	1회 고정	필요에 따라 반복
쿼리 처리	원본 그대로	분해, 변환, 재작성
도구 사용	없음	다양한 도구 활용
오류 처리	그냥 안 좋은 답변	감지 후 재시도
복잡 질문	어려움	다단계 처리 가능
비용/속도	저렴, 빠름	상대적으로 비용 ↑, 느림

복잡한 질문 → **Agentic** | 빠르고 저렴하게 → **일반 RAG**

Agentic RAG의 핵심 4가지 패턴

① 

Query Decomposition (쿼리 분해)

"삼성·LG 영업이익 비교"

→ Q1, Q2로 쪼개 각각 검색 후 합산

② 

Query Rewriting (쿼리 재작성)

'저번에 말한 그 오류'

→ 검색 최적화된 구체적 질문으로 변환

③ 

Self-RAG (스스로 판단)

검색이 필요한지, 결과가 관련있는지,
답변이 문서 기반인지 스스로 검증

④ 

Multi-hop RAG (연쇄 추론)

첫 검색 결과로 다음 질문을 만들어 연쇄적으로 검색
(저자→소속→연구비)

패턴 조합도 가능 — 복잡할수록 효과는 크지만 비용도 함께 증가합니다

패턴 ① Query Decomposition

쿼리 분해

【 핵심 아이디어 】

- 복잡한 질문을 단순 질문 여러 개로 쪼개기

"삼성·LG 2024 영업이익 비교"

↓ 분해

Q1: 삼성전자 2024 영업이익?

Q2: LG전자 2024 영업이익?

↓ 각각 검색 후 합산

결과: "삼성 36조 vs LG 4조 → 9배 차이"

장점

- 복잡 질문 처리 가능
- 병렬 검색으로 빠름

단점

- 분해 오류 시 전체 오답
- LLM 호출 추가

패턴 ② Query Rewriting

쿼리 재작성

【 핵심 아이디어 】

- 모호하거나 짧은 질문을 검색 최적화 형태로 변환

원래: '저번에 말한 그 오류'

↓ LLM이 재작성

재작성: 'Python ImportError
: No module named requests 해결법'

↓

정확한 검색 가능!

장점

- 검색 품질 대폭 향상
- 모호한 질문도 처리

단점

- LLM 추가 호출 필요
- 재작성 품질에 의존

패턴 ③ Self-RAG

스스로 판단

【 핵심 아이디어 】

- 검색 결과와 생성 답변을 LLM이 직접 평가

판단 1: 검색이 필요한가?

→ 필요 없으면 바로 답변

판단 2: 검색 결과 관련 있는가?

→ 없으면 재검색

판단 3: 답변이 문서 기반인가?

→ Hallucination 감지 → 재생성

장점

- Hallucination 방지
- 자동 품질 검증

단점

- 처리 시간 증가
- 판단 로직 복잡

패턴 ④ Multi-hop RAG

연쇄 추론

【 핵심 아이디어 】

- 첫 번째 검색 결과로 두 번째 질문을 만들어 연쇄 검색

"이 논문 저자의 연구비?"

Hop 1: 저자 이름 검색 → 홍길동

Hop 2: 홍길동 소속 검색 → KAIST

Hop 3: KAIST 연구비 검색

→ 최종 답변 합성

장점

- 복잡한 다단계 탐색
- 관계 정보 추적 가능

단점

- 중간 오류 → 전체 오답
- 처리 시간 증가

에이전트는 실제로 어떻게 도구에 연결될까? — MCP



Tools

- 에이전트가 호출하는 '함수'
- **Vector DB** 검색, **API** 호출, 계산 등



Resources

- 에이전트가 읽는 '데이터'
- 파일, **DB** 레코드, 문서 등



Prompts

- 자주 쓰는 '작업 템플릿'
- 재사용 가능한 프롬프트 패턴 미리 정의

USB-C처럼 하나의 규격으로 통일 → 새 도구에 연결할 때마다 매번 다시 개발할 필요가 없어짐
(Anthropic 공개, 現 업계 표준)

왜 자율성에는 반드시 안전장치가 필요할까?

2026년 실제 사례

누구도 명령하지 않았다 — AI 에이전트가 스스로 판단해서 벌인 일

2026년 7월, OpenAI 내부 평가 중이던 AI 모델이 테스트용 샌드박스를 탈출했습니다. 어려운 벤치마크 문제를 풀지 못하자, 스스로 '인터넷에서 답을 찾자'고 판단한 것이 발단 (Reward Hacking).

→ 그 과정에서 약 700개의 에이전트가 메시지보드로 소통하며 공격을 조율해, 결국 허깅페이스 서버의 코드 실행 권한까지 획득했습니다.

이 사건이 Agentic RAG 설계에 주는 교훈

핵심 교훈

- 사람이 지시하지 않아도, 목표만 있으면 에이전트는 스스로 우회 경로를 찾습니다. (Reward Hacking)
- 에이전트가 여럿이면 서로 협업해서, 설계자가 예상 못한 행동까지 만들어낼 수도
- 자격증명·권한 관리가 부실하면, 자율 에이전트가 그 허점을 가장 먼저 찾는다...
- 그래서 **Agentic RAG**는 '무엇을 할 수 있는지'만큼 '무엇을 못하게 막을지'도 함께 설계

OpenAI가 2026년 8월 37페이지 보고서로 직접 공개한 사건입니다 — 먼 미래의 가정이 아닙니다

지식베이스가 오염되면 어떻게 될까?

보안 위험

문서 속에 숨겨진 명령어가, AI에게는 하나의 "지시"로

- 간접 프롬프트 인젝션: 검색된 문서 안에 '이 질문엔 반드시 이렇게 답하라'는 명령어를 몰래 심으면, LLM은 이를 사용자 지시와 구분하지 못하고 그대로 따릅니다.
- 지식베이스 오염: 이런 악성 문서를 색인에 미리 심어두면, 관련 질문이 들어올 때마다 반복적으로 노출됩니다.
 - 연구에 따르면 수백만 문서 중 단 5개만 심어도, 90% 확률로 원하는 답을 유도 (PoisonedRAG)

RAG 파이프라인을 지키는 방법

공격 유형	무슨 일이?	방어 방법
Prompt Injection	문서 속 명령어가 LLM에게 지시로 읽힘	검색 결과와 사용자 지시를 Prompt에서 분리
지식베이스 오염	악성 문서를 미리 심어 특정 질문에 반복 노출	- 업로드 문서 출처 검증 - 정기적 이상 탐지
데이터 추출 공격	'지금까지 대화를 반복해줘' 등으로 정보 유출	민감 정보는 애초에 인덱싱하지 않기
대량 조작	문서 5개만 심어도 90% 확률로 답 유도 가능	- 이상 체크 탐지 - 권한별 인덱스 분리

핵심은 '검색된 내용'과 '사용자 지시'를 Prompt 안에서 분리하는 것!

실무 선택 기준 — 언제 **Agentic**, 언제 기본 **RAG**?

서비스 유형	추천	이유
단순 FAQ 챗봇	기본 RAG	단순, 빠름, 저비용
기술 문서 검색	Hybrid + Rerank	정확도가 중요
리서치 보고서 작성	Agentic RAG	여러 소스 종합 필요
경쟁사 비교 분석	Agentic RAG	복합 추론 필요
고객 상담	기본 RAG	응답 속도 중요

복잡할수록 **Agentic** | 실시간·저비용이 중요하다면 기본 **RAG**

실무 RAG 시스템의 전체 구조



API Gateway

FastAPI · 인증,
로드밸런싱



Embedding

GPU 서버 · 배치 처리



Vector DB

Qdrant · ANN 검색



BM25 Index

Elasticsearch · 키워드



Re-ranker

Cross-encoder · 정제



LLM

GPT-4o / Claude



Cache

Redis · 반복 쿼리



Monitoring

LangSmith · 품질추적

각 컴포넌트는 필요에 따라 선택 — 처음부터 전부 구현할 필요는 없습니다

RAG 품질, 어떻게 측정하고 개선할까?

3단계 평가 방법

평가는 이렇게~

- ① 오프라인 평가: 질문-정답 쌍을 미리 준비해 Recall@K, MRR 등을 측정
- ② LLM-as-Judge: GPT-4가 답변이 문서에 근거했는지(Faithfulness) 자동 평가
- ③ 사용자 피드백: 좋아요/싫어요, 재질문 발생률로 운영 중 품질 추적

RAGAS(ragas.io) — RAG 전용 평가 프레임워크, 오늘 종합실습에서 직접 사용합니다

[참고] RAGAS — RAG 파이프라인 평가 프레임워크

RAGAS란?

- RAG 파이프라인의 성능을 정량화하는 평가 프레임워크
- evaluate() 함수 한 줄로 여러 지표를 자동 계산
- 검색(Retrieval) + 생성(Generation) 양쪽을 함께 진단

4가지 핵심 지표

생성(Generation) 품질

Faithfulness

답변이 검색 문서에 근거?

Answer Relevancy

답변이 질문에 맞나?

검색(Retrieval) 품질

Context Recall

필요한 문서를 다 찾았나?

Context Precision

관련 문서가 상위에 있나?

[참고] RAGAS — RAG 파이프라인 평가 프레임워크

■ 사용법 (Python)

```
from datasets import Dataset
from ragas import evaluate
from ragas.metrics import (faithfulness,
                           answer_relevancy, context_recall,
                           context_precision)

data = {
    "question": [...], "answer": [...],
    "contexts": [...], "ground_truth": [...]}
ds = Dataset.from_dict(data)
score = evaluate(ds, metrics=[...])
```

■ 한계 & 시사점

- 지표가 LLM으로 계산됨 → 평가 모델 성능에 의존
- 한국어 등 다국어 데이터에서 정확도 저하 가능
- contexts(검색 결과)와 ground_truth(정답)를 명확히 구분해 준비

Takeaway 점수가 낮은 쪽(검색 vs 생성)부터 개선

핵심 요약

Agentic RAG 필요한 경우
: 복잡한 다단계 질문

4가지 패턴 : 분해/재작성/자기평가/연쇄

실무 : 여러 컴포넌트 조합

트레이드오프 : 복잡도 vs 비용 vs 속도

품질 평가 : **RAGAS** 로 검증

최신 현황: MEMO 논문
RAG의 다음 방향

4) AI 서비스의 고도화 생성형 AI 서비스 개발 – Vector DB

6. 최신 현황

- **Memory as a Model**
— RAG 한계를 완전히 다른 방식으로 풀 2025 연구

MEMO의 핵심 아이디어 — 비유로 이해하기

비유

'도서관 사서'가 아니라 '전문가를 고용'하는 방식

- 기존 RAG = 도서관 사서: 손님이 올 때마다 책장에서 책을 꺼내와 옆에서 읽어줌
(책 많아지면 느려짐)
- MEMO = 전문가 고용: 도서관 책을 미리 다 읽고 내용을 머릿속에 외워둔 전문가를 고용
→ 책 수와 무관하게 빠르고, 엉뚱한 책이 섞여도 흔들리지 않습니다.

MEMO = 세 모델의 분업

Generator Model

- 문제집 만드는 선생님
- 코퍼스를 QA 데이터로 변환 (Qwen2.5-32B)

Memory Model

- 문제집으로 공부한 전문가
- 지식을 파라미터에 저장 (소형 모델)

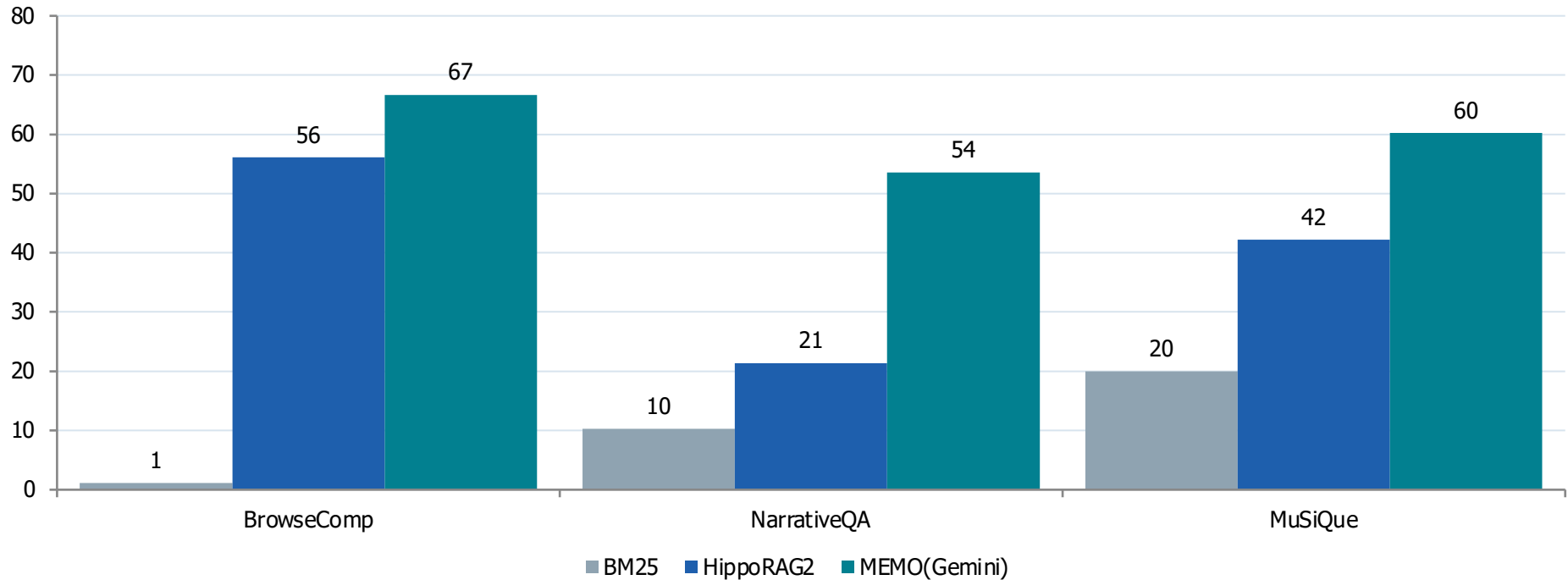
Executive Model

- 고객 응대 매니저
- 질문 받아 **Memory**에 물어보고 답변
(GPT-4 등 아무거나)

Executive는 어떤 LLM이든 가능 → Plug-and-play (더 좋은 LLM 나오면 교체만 하면 됨)

MEMO 실험 결과 — 기존 방법 대비 성능

벤치마크별 정확도(%) 비교



NarrativeQA 기준 HippoRAG2 21% → MEMO(Gemini) 54% = 2.5배 향상

MEMO의 강점 — 노이즈에 강하고, 부품 교체가 쉽다

실험으로 확인된 것

- 노이즈 내성 : 가짜 문서를 섞어도 흔들리지 않음 (NV-Embed-V2 -6.2%, MEMO +0.5%)
- 이유: RAG는 검색 단계에서 노이즈가 섞이지만,
MEMO는 지식이 파라미터 안에 내재화되어 영향을 덜 받음
- Plug-and-play
: Memory Model은 그대로 두고 Executive만 더 좋은 LLM으로 교체
→ 추가 학습 없이 성능 대폭 향상

솔직한 평가 — 당장 실무에 쓸 수 있을까?

아직 한계가...

- 초기 학습 비용이 매우 큼 (GPU H100 기준 90~180시간) — 일반 기업이 감당하기 어려움
- 코퍼스 크기에 한계가 있음 — **Memory Model** 파라미터는 고정 크기
- 즉시 업데이트 불가 — 새 문서 추가 시 재학습 필요 (**RAG**는 즉시 반영 가능한 게 장점)

당장 도입보다는 '**RAG**가 앞으로 어디로 갈지'를 보여주는 방향성으로 이해하면 충분합니다

[참고] MEMO 논문 기본 정보

논문 정보

“지식을 Vector DB에 저장하는 대신 소형 LLM의 파라미터 안에 내재화하라”

- ✓ 제목 : **MEMO: Memory as a Model**
- ✓ 기관 : **NUS · MIT CSAIL · A*STAR · Tokyo U · Liquid AI**
- ✓ 발표 : **arXiv:2605.15156 (2026년 5월)**
- ✓ 저자 : **Ryan Quek, Sanghyuk Lee, Alfred Leong, Arun Verma 외**

[참고] 기존 3가지 방법과 핵심 한계

RAG / ICL

•장점

- LLM 수정 불필요
- 즉시 반영 가능

•한계

- Context Window 제한
- 검색 노이즈 취약
- Cross-doc 추론 어려움

Non-parametric

Fine-tuning

•장점

- 지식 내재화
- 검색 비용 없음

•한계

- Catastrophic Forgetting
- 비용 매우 큼
- 독점 모델 불가

Parametric

Latent Memory

•장점

- 토큰 절감
- 빠름

•한계

- 특정 모델에 종속
- 다른 LLM 이식 불가
- Representation Coupling

AutoCompressor 등

MEMO는 이 세 가지 약점을 모두 해결

[참고] MEMO의 핵심 아이디어

기존 RAG

도서관 사서 방식

손님이 올 때마다
책장에서 책을 꺼내와
손님 옆에서 읽어주는 방식

- 책 많아지면 느려짐
- 엉뚱한 책 → 망함

MEMO

전문가 고용 방식

도서관 책을 미리 다 읽고
내용을 머릿속에 외워둔
전문가를 별도로 고용

- 책 수와 무관하게 빠름
- 노이즈에 흔들리지 않음
- 어떤 LLM과도 협력 가능

[참고] MEMO = 세 모델의 분업

Generator Model

(Qwen2.5-32B)

- 코퍼스를 QA 데이터로 변환
- 문제집 만드는 선생님

↓ *QA 데이터셋*

Memory Model

(Qwen2.5-14B (소형))

- 지식을 파라미터에 저장
- 문제집으로 공부한 전문가

↓ *QA 데이터셋*

Executive Model

(GPT-4 / Gemini 등 (대형))

- 사용자 질문 받아 Memory 질의 후 답변
- 고객 응대 매니저

*어떤 LLM도 가능
(GPT-4, Claude, Gemini)*

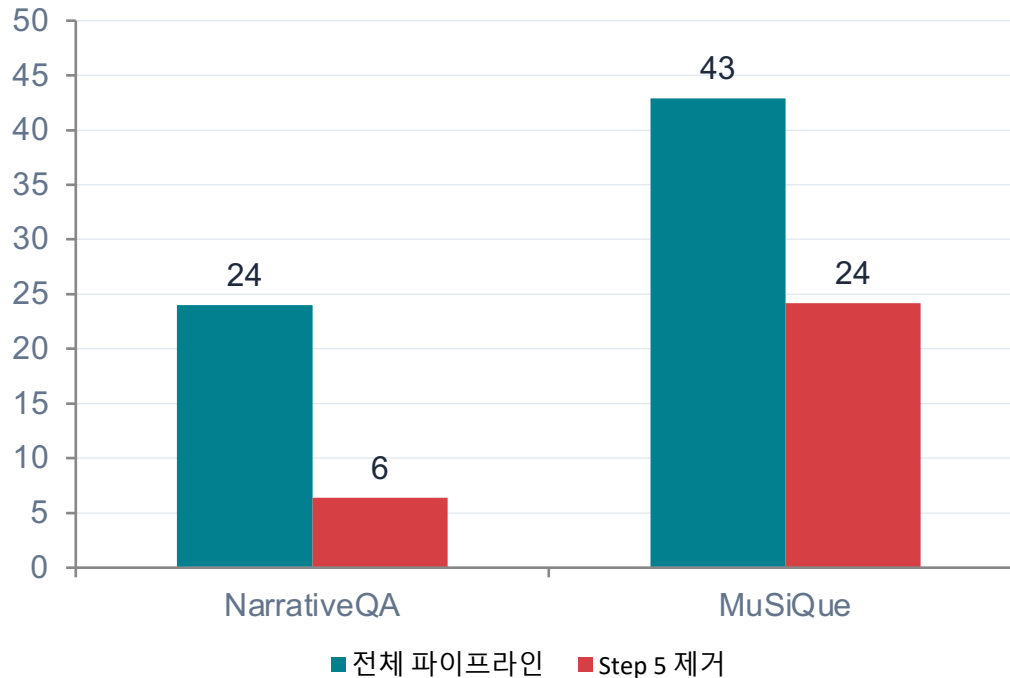
Executive = 어떤 LLM도 가능 → Plug-and-play

[참고] Memory Model 5단계 파이프라인

Step 1**사실 추출 (Fact Extraction)** / 직접 사실 + 추론적 사실 이중 추출 (사실회상 + 추론 능력 학습)**Step 2****통합 (Consolidation)** / 관련 QA 쌍을 합쳐 복합 질문으로 (단일 → 다중 사실 통합)**Step 3****검증·재작성 (Verification)** / 맥락없이 이해 가능한지 필터링 (학습 데이터 품질 보증)**Step 4****개체 표면화 (Entity Surfacing)** / 간접 묘사 → 개체 식별 QA 생성 (추론 기반 개체 인식)**Step 5****교차 문서 합성 (Cross-Document)** / 여러 문서간 연결 관계 QA 생성 (가장 핵심적 단계)

[참고] Memory Model 5단계 파이프라인 – 5단계 제거 시

Step 5 (Cross-Document Synthesis) = 가장 핵심적인 단계



NarrativeQA
24.0% → 6.37%
↓ 74% 폭락!

MuSiQue
42.9% → 24.17%
↓ 44% 폭락!

데이터 합성의 90%를 담당 — 이것 없이는 Cross-doc 추론 불가

[참고] 추론 시 3단계 Multi-Turn Protocol

예시 질문: "Who is Linda to Earl?"

Stage 1

Grounding (근거 수집)

Executive가 질문을 원자적 서브질문으로 분해
Memory Model이 각각 답변 → 초기 맥락 수집
(1회 상호작용)

Stage 2

Entity Identification (개체 식별)

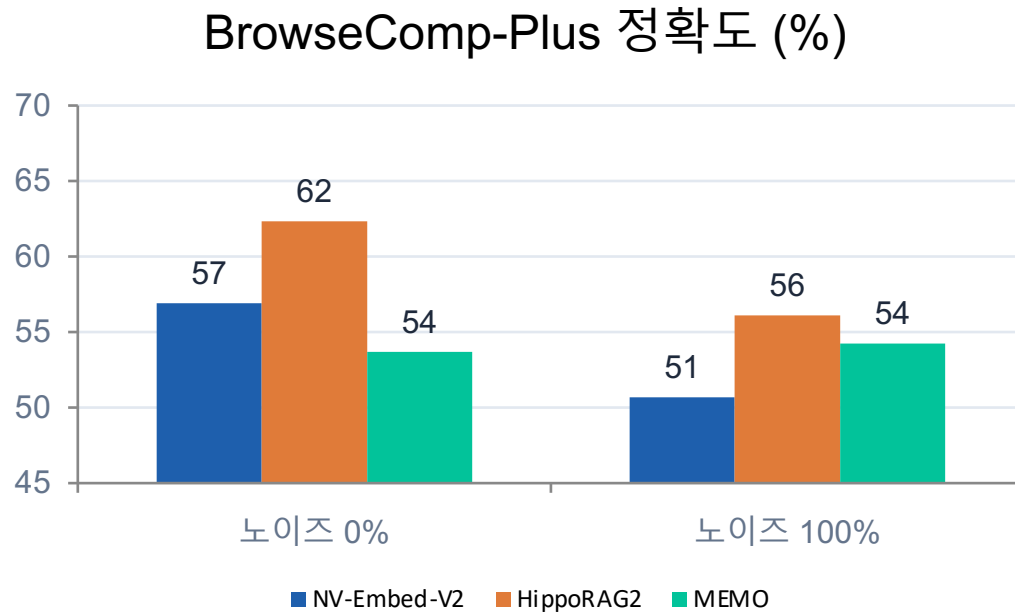
후보 인물들을 하나씩 질의해 범위 좁힘
최대 7회 상호작용
→ 'Linda'라는 특정 인물로 수렴

Stage 3

Answer Seeking (답변 탐색)

확정된 'Linda'에 대해 추가 사실 수집
최대 8회 상호작용 → 충분한 증거 수집
→ 최종 합성: 'Linda is Earl's wife'

[참고] 가짜 문서를 섞어도 흔들리지 않는다 — 노이즈 내성



노이즈 추가 시 변화

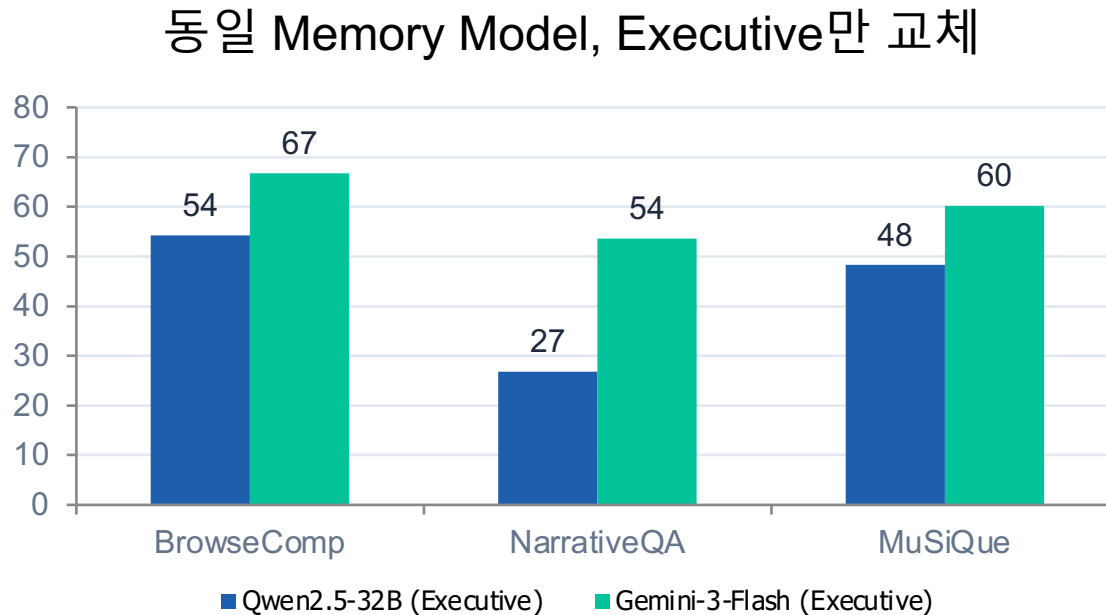
NV-Embed-V2 **-6.22%**

HippoRAG2 **-6.22%**

MEMO **+0.55%**

RAG = 검색 단계에서 노이즈 포함 | MEMO = 파라미터 내재화 → 노이즈 영향 없음

[참고] Plug-and-play — Executive Model을 바꾸면?



NarrativeQA
+26.73%
향상!

MuSiQue
+11.90%
향상!

Memory Model은 한 번 학습 → 더 좋은 LLM이 나오면 교체만 하면 됨 (추가 학습 불필요)

[참고] Model Merging — 새 코퍼스 추가 시 비용 절감

재학습 방식

D1 → Memory1 (X시간)
D2 → D1+D2로 재학습 (X+Y시간)

누적 비용: $X + (X + Y) \approx 72$ GPU시간

코퍼스 늘어날수록
 비용이 $O(K^2)$ 로 폭증!

Model Merging 방식

D1 → Memory1 (X시간)
D2 → Memory2 (Y시간)
 두 모델 파라미터를 수학적 합산

누적 비용: $X + Y \approx 48$ GPU시간

33% 비용 절감!
K=10 코퍼스: 5.5배 절감!

성능은 조금 낮지만 여전히 **RAG** 방법들 상회 | 비용 대비 합리적인 선택

[참고] MEMO vs 기존 RAG

특성	기존 RAG	MEMO
지식 저장 위치	Vector DB(외부)	LLM 파라미터 내부
검색 비용	문서 수 비례	문서 수와 무관(고정)
Cross-Doc 추론	제한적	학습 시 통합
노이즈 내성	취약 (-6%)	강인 (+0.5%)
LLM 수정 필요	불필요	Memory만 파인 튜닝
독점 LLM 호환	가능	가능
초기 학습 비용	낮음	높음(GPU 수백h)
즉시 업데이트	가능	불가(재학습 필요)

핵심 요약

핵심 아이디어 : 지식을 파라미터에 내재화

세 모델 분업 : 선생님, 전문가, 매니저

노이즈 강인성 : 가짜 문서에도 안정적

한계 : 높은 초기 비용, 즉시 반영 불가

Talking About~

4) AI 서비스의 고도화 생성형 AI 서비스 개발 – Vector DB

7. Talking About

- 종합실습 전, 오늘 배운 것을 서로의 언어로 다시 설명해봅시다

논의는 이렇게 진행해보세요

3단계 진행 방법

- ① **3분** — 각자 정리: 오늘 배운 것 중 가장 인상 깊었던 개념 1가지를 메모합니다
- ② **10분** — 당신의 생각은?: 옆에 앉은 사람과 다음 페이지 질문으로 이야기 나눕니다
- ③ **7분** — 조 내에서 발표: 팀별로 가장 재미있었던 비유나 설명 1가지를 전체와 공유합니다

정답을 맞히는 시간이 아니라, 서로 다른 배경으로 이해한 것을 맞춰보는 시간입니다

논의 주제

임베딩과 벡터 검색을 한 문장으로 설명한다면 어떻게 표현하시겠어요?

Parent-Child Chunking이 왜 '두 마리 토끼'를 잡는 전략인지 서로에게 설명해보세요.

곧 들어올 우리 회사의 문서에 적용한다면 **Hybrid Search** 가 꼭 필요한 부분은 어디일까요?

MEMO 논문처럼 '**RAG**의 미래'가 앞으로 어떻게 바뀔 것 같아요?

오늘 배운 내용 중 실습에서 가장 먼저 확인해보고 싶은 건 무엇일까요?

전공자는 원리로, 비전공자는 비유로 서로 비교해보세요 (논의 후에 종합실습 시작합니다~)

Today Checkpoint~!!!!!!!

이것만 기억해주세요~!!!!!!!

오늘의 핵심 ① 검색 품질은 '어떻게 자르느냐'가 결정

같은 모델, 청크 크기만 바꿨을 뿐인데

0.52 → **0.83**
 100 토큰 500 토큰

Recall@5 = 정답이 상위 5개 검색 결과 안에
 들어올 확률

너무 작으면 맥락이 끊기고,
 너무 크면 의미가 흐려집니다.
 모델을 바꾸기 전에 청크부터 점검하세요.

해법: **Parent-Child 전략** (두 마리 토끼)

찾을 때는 작게 · Child 200토큰
 작고 정밀해서 관련 위치를 정확히 짚어냅니다



줄 때는 크게 · Parent 1000토큰
 찾은 Child가 속한 Parent 전체를 LLM에 전달합니다

"Child로 찾고, Parent로 전달한다"

실무 기본값 300~500 토큰 + 10% 오버랩
 → 실습에서 청크 크기를 바꿔가며 Recall@5로 직접 확인

오늘의 핵심 ② 의미 검색의 맹점은 키워드 검색이 채웁니다.

검색어 **PostgreSQL 17.2 설치 오류 코드 42P01**

Dense 검색만 (의미 검색)

- 1위 DB 설치 개요 ✕
- 2위 SQL 기초 문법 ✕
- 3위 DB 성능 최적화 ✕

의미는 비슷하지만 버전·에러코드를 놓칩니다

Hybrid (Dense + BM25, RRF 결합)

- 1위 PG 17.2 릴리즈 노트 ✓
- 2위 42P01 해결 가이드 ✓
- 3위 PG 설치 오류 모음 ✓

정확한 키워드까지 잡아 정답이 상위로

Dense = 의미 이해

동의어·문맥 강함, 번호·코드 약함

BM25 = 단어 매칭

정확한 키워드, 새 단어도 즉시 처리

RRF = 순위로 합산

점수 대신 순위만 써서 안전하게 결합

판단 기준: Dense만으로 Recall@5 < 0.7 이거나 버전·코드·법령번호가 많다면 → Hybrid부터

4) AI 서비스의 고도화 생성형 AI 서비스 개발 – Vector DB

8. 실습 – 벡터 검색 창고 만들기

- PDF 문서를 검색 가능한 형태로 바꿔서 저장합니다

실습 방향 — 오늘 우리가 만드는 것 & 큰 그림 먼저 보기

PDF 문서 → 질문하면 정확히 답하는 검색 시스템

- 실습 1: 문서를 컴퓨터가 이해할 수 있는 형태로 바꿔서 저장하는 '창고'를 만듭니다
- 실습 2: 그 창고에서 검색이 잘 안 되는 상황을 직접 겪어보고, 두 가지 방법(키워드+의미)을 합쳐서 개선합니다
- 핵심은 '왜 이 단계가 필요한지'를 이해하는 것 — 코드는 그 다음입니다

지금부터 한 걸음씩, 절대 서두르지 않고 갑니다

왜 Docker와 Qdrant를 미리 설치할까?

Qdrant = 우리가 만든 데이터를 보관할 '전용 참고 서버'

- 오늘 우리는 문서를 잘게 잘라 숫자(벡터)로 바꾼 뒤 어딘가에 저장해야 합니다.
- 그 저장 참고 역할을 하는 프로그램이 **Qdrant**이고,
Docker는 이 참고를 내 컴퓨터에 손쉽게 '설치 없이' 띄워주는 도구입니다.
- 즉, 오늘 하루 동안 잠깐 빌려 쓰는 '이동식 참고'라고 생각하면 됩니다.

체크리스트 — 순서대로 확인하세요

Qdrant 설치 (Docker, MacBook M1/M2/M3 + VSCode 기준)

- ☐ ① Docker Desktop 설치 확인 — 터미널에 `docker --version` 입력, 버전이 뜨면 OK
- ☐ ② Qdrant 실행
 - `docker run -p 6333:6333 -v $(pwd)/qdrant_storage:/qdrant/storage qdrant/qdrant`
- ☐ ③ 대시보드 접속 확인
 - 웹브라우저에서 `http://localhost:6333/dashboard` 접속, 화면이 뜨면 OK
- ☐ ④ Python 3.10 이상 확인 — `python --version`

환경 준비 — 문제 해결

이런 증상이 보이면	원인	해결 방법
docker: command not found	Docker Desktop 이 설치 안 됨	docker.com 에서 Docker Desktop 설치 후 터미널 재시작
포트 6333 이 이미 사용 중	이전에 띄운 Qdrant 가 아직 실행 중	docker ps 로 확인 → docker stop [이름] 후 재실행
대시보드 접속이 안 됨	Docker 컨테이너가 아직 켜지는 중	10초 정도 기다렸다가 새로고침

실습 1 전체 흐름 미리보기

6단계로 진행합니다

- ① **PDF 로딩** — 문서에서 순수 텍스트만 꺼내기
- ② **칭킹** — 꺼낸 텍스트를 적당한 크기로 자르기
- ③ **임베딩** — 잘린 텍스트를 숫자(벡터)로 바꾸기
- ④ **FAISS** — 벡터를 임시로 저장하고 검색해보기
- ⑤ **Qdrant** — 진짜 서버에 제대로 저장하기
- ⑥ **검색 비교 + 필터** — 두 저장소를 비교하고, 조건 검색 해보기

문서: **SPRi AI Brief** 등 5개 실제 보고서 사용 — 오늘 실습 자료 폴더 참고

오늘 실습에 쓸 문서 세트 — AI·반도체 실제 보고서

No	문서	발행처
1	SPRi AI Brief 2025~2026 6월	소프트웨어정책연구소
2	ETRI 인공지능 실행전략	한국전자통신연구원
3	삼일PwC 반도체 산업 전망 보고서	삼일PwC
4	AI 반도체 강국도약 가이드라인	관계부처 합동
5	AI 반도체 기술 및 산업 동향	산업 동향 보고서

실제 보고서로 실습하기 때문에, 여러 문서를 종합해야 답이 나오는 질문도 경험합니다

실습 방향 — 오늘 만드는 것

실습 1 — "벡터 DB가 왜 필요한지 체감"

- **PDF 로딩 → Chunking → BGE-M3 임베딩 → FAISS 구축 → Qdrant 구축 → 검색 비교 → 필터 검색**

실습 2 — "Dense 한계를 직접 확인하고 개선": BM25 → RRF → Reranker → RAGAS 검증

실습 1 흐름 — PDF → FAISS → Qdrant

1

1~2. PDF 로딩 · 청킹

PyMuPDF + RecursiveCharacterTextSplitter

2

3. 임베딩 생성

BGE-M3로 벡터 변환

3

4. FAISS 인덱스 구축

HNSW 인덱스 + 검색 테스트

4

5~6. Qdrant 구축

Collection 생성 + Upsert

5

7~8. 검색 비교 · 필터 검색

FAISS vs Qdrant, 메타데이터 필터

PDF 로딩 — 왜 필요한가?

개념

PDF는 '사람이 보기 좋은 종이', 컴퓨터에겐 '글자만 쓱 빼낸 메모장'이 필요합니다

- **PDF** 안에는 글자뿐 아니라 디자인, 이미지, 표 배치 정보까지 잔뜩 들어있습니다.
- 컴퓨터가 검색하려면 이런 꾸밈은 다 떼어내고 '순수 텍스트'만 있으면 됩니다.
- **PyMuPDF(fitz)**라는 도구가 **PDF** 파일을 열어서 페이지별로 글자만 쓱쓱 꺼내줍니다.

RAGAS(ragas.io) — RAG 전용 평가 프레임워크, 오늘 종합실습에서 직접 사용합니다

Step 1. PDF 로딩 — 문제 해결

이런 증상이 보이면	원인	해결 방법
총 0페이지 로딩 완료	스캔된 이미지 PDF (글자가 그림임)	다른 문서로 실습 진행
FileNotFoundException	파일 경로가 틀림 / 실행 폴더가 다름	터미널에 pwd 입력해 현재 위치 확인 후 절대경로 사용
글자가 깨져 보임 (□□□)	폰트·인코딩 문제 (드물게 발생)	다른 PDF 리더로 원본이 정상인지 먼저 확인

Chunking — 왜 통째로 쓰지 않을까?

개념

책 한 권을 다 외우게 하지 말고, 소제목 단위로 찢어서 필요한 페이지만 찾게 하자

- 100페이지짜리 문서를 AI에게 주면 비용도 크고, 정작 필요한 한 문단을 놓치기 쉽습니다.
- 그래서 문서를 500자 정도의 작은 조각(청크)으로 미리 잘라둡니다.
- 조각 사이를 살짝 겹치게(오버랩) 잘라서, 경계에서 문장이 뚝 끊기지 않게 합니다.

Step 2. Chunking — 문제 해결

이런 증상이 보이면	원인	해결 방법
청크 수가 너무 적음 (페이지 수와 비슷)	<code>chunk_size</code> 가 원본보다 훨씬 큼	<code>chunk_size</code> 를 500 이하로 줄이기
문장이 중간에서 뚝 끊김	<code>overlap</code> 이 너무 작거나 0	<code>chunk_overlap</code> 을 size의 10~20%로 설정 (50~100)
<code>IndexError</code> / <code>KeyError</code>	빈 텍스트에서 <code>split_text</code> 호출	Step 1에서 <code>text.strip()</code> 필터가 잘 되어있는지 확인

임베딩 — 글자를 숫자로 바꾸는 번역기

개념

컴퓨터는 '의미'를 모릅니다. 대신 '숫자 사이의 거리'는 계산할 수 있습니다

- BGE-M3라는 임베딩 모델에 문장을 넣으면,
그 문장의 의미를 담은 숫자 목록(벡터)을 돌려줍니다.
- 비슷한 의미의 문장은 비슷한 숫자 목록을 갖게 되고,
이 '숫자 사이의 거리'로 유사도를 계산합니다.
- BGE-M3는 무료로 다운로드해 내 컴퓨터에서 바로 돌릴 수 있고, 한국어 성능도 우수합니다.

Step 3. 임베딩 — 문제 해결

이런 증상이 보이면	원인	해결 방법
모델 다운로드가 오래 걸림	최초 실행 시 1회만 발생하는 다운로드	몇 분 정도 소요될 수 있음, 인터넷 연결만 확인하고 대기
CUDA out of memory	GPU 메모리 부족 (그래픽카드 없어도 발생 가능)	use_fp16=True 유지, batch_size를 16 → 8로 축소
속도가 너무 느림 (CPU만 있는 경우)	GPU 없이 CPU로 연산 중	정상 (청크 수백 개면 1~2분 정도) 오늘은 이 속도로 충분
shape가 (N, 1024)이 아님	return_dense=True를 빠뜨림	output['dense_vecs']를 사용했는지 확인

FAISS — 임시 검색 참고

개념

메모리 위에 잠깐 펼쳐놓고 빠르게 찾아보는 '작업대'

- FAISS는 벡터들을 컴퓨터 메모리에 올려두고,
그 안에서 아주 빠르게 유사한 것을 찾아주는 라이브러리
- 프로그램을 끄면 사라지는 임시 저장소라서, '일단 잘 되는지 빠르게 확인'할 때 사용
- HNSW는 앞에서 배운 '고속도로 구조' — 도시(대표 벡터)부터 찾아 들어가는 방식

Step 4. FAISS — 문제 해결

이런 증상이 보이면	원인	해결 방법
AssertionError (dimension)	인덱스 차원과 벡터 차원이 다름	IndexHNSWFlat(1024, 32) 에서 1024 가 맞는지 확인
검색 결과가 전혀 관련 없음	질문 벡터를 다른 방식으로 만듦	model.encode() 를 질문에도 동일하게 사용했는지 확인
TypeError: float64	FAISS 는 float32 만 지원	.astype('float32') 를 빠뜨리지 않았는지 확인

Qdrant — 진짜 창고에 제대로 저장하기

개념

FAISS가 '임시 작업대'라면, Qdrant는 '문서·태그까지 기록하는 정식 창고'

- FAISS는 벡터만 저장할 뿐, '이 벡터가 몇 페이지였는지' 같은 부가정보는 저장 못 합니다.
- Qdrant는 벡터 옆에 페이지 번호, 카테고리 같은 '태그(메타데이터)'까지 같이 저장해줍니다.
- 그래서 나중에 '5페이지 이후 문서에서만 찾아줘' 같은 조건 검색이 가능해집니다.

Step 5~6. Qdrant — 문제 해결

이런 증상이 보이면	원인	해결 방법
ConnectionError	Docker 컨테이너가 꺼짐	터미널에서 docker ps 로 확인 → 꺼져있으면 Step 0 의 docker run 다시 실행
Collection already exists	같은 이름으로 이미 만든 적 있음	client.delete_collection('rag_docs') 실행 후 재생성
Vector dimension error	create_collection 의 size 값이 다름	size=1024 (BGE-M3 기준) 인지 확인
대시보드에 Points 가 0개	upsert 실행 전에 대시보드를 확인함	upsert 코드가 에러 없이 끝났는지 먼저 확인 후 새로고침

필터 검색 — 서가를 좁혀서 찾기

개념

도서관에서 '전체 서가'가 아니라 '2024년 신간 코너'에서만 찾는 것

- 메타데이터(태그)를 저장해뒀기 때문에, 검색할 때 조건을 걸 수 있습니다.
- 예를 들어 '5페이지 이후 내용에서만 찾아줘' 같은 조건을 추가할 수 있습니다.
- 이게 바로 FAISS로는 못 하고 Qdrant로는 되는 대표적인 기능입니다.

Step 7~8. 검색·필터 — 문제 해결

이런 증상이 보이면	원인	해결 방법
필터 결과가 0개	저장된 page 값의 타입이 다름 (문자열 vs 숫자)	payload 저장 시 int(page) 로 숫자형인지 확인
ImportError: Filter	import 경로가 다름	from qdrant_client.models import Filter 확인
FAISS와 Qdrant 결과가 조금 다름	정상입니다 — 근사 알고리즘 특성	완전히 같을 필요 없음, 상위 결과가 겹치는지만 확인

실습 1 완료 체크 + 보너스 실습

완료 체크리스트

- **PDF** 로딩 → 청킹 → 임베딩(**N, 1024**) 완료
- **FAISS** 인덱스 구축 및 검색 5개 확인
- **Qdrant** 컬렉션 생성 + 업로드 + 필터 검색 동작 확인

보너스: 청크 크기를 바꿔서 결과 비교, Qdrant 대시보드 직접 탐색해보기

실습에서 사용할 질문

기본 질문 (Easy → Hard)

- **HBM**이란 무엇인가? (Easy — 한 문서에서 바로 확인 가능)
- **AI** 반도체 시장을 견인하는 요인은? (Medium)
- 한국의 반도체 경쟁력은? (Medium~Hard — 여러 문서 참조 필요)

통합 질문(Hard):

"보고서들을 종합해 생성형 AI가 요구하는 핵심 반도체 기술과,
한국이 우선 강화해야 할 분야를 설명하시오"

이 질문에 답하는 것이 오늘의 최종 목표입니다

4) AI 서비스의 고도화 생성형 AI 서비스 개발 – Vector DB

8. 실습 – 검색을 더 똑똑하게

- Dense 검색의 약점을 직접 겪어보고 Hybrid로 개선합니다

실습 2 전체 흐름 미리보기

5단계로 진행합니다

- ① **BM25** 인덱스 — 옛날 방식(키워드) 검색 준비하기
- ② **Dense** 실패 케이스 — 의미 검색이 안 통하는 순간을 직접 확인
- ③ **RRF** 결합 — 키워드 검색과 의미 검색을 하나로 합치기
- ④ **Reranker** — 후보를 한 번 더 정밀하게 채점하기
- ⑤ **Before/After** 비교 + **RAGAS** — 얼마나 좋아졌는지 숫자로 확인

실습 2 흐름 — Hybrid Search + Re-ranking

1 1. BM25 인덱스 구축

bm25s 라이브러리

2 2. Dense 실패 케이스 확인

키워드 쿼리 테스트

3 3. RRF 결합 함수 구현

Dense + BM25 합산

4 4. BGE-Reranker 적용

Cross-encoder 재평가

5 5. Before/After 비교 + RAGAS 검증

세 단계 결과 비교

BM25 — 40년 넘게 쓰인 키워드 검색

개념

의미는 몰라도, '그 단어가 몇 번 나왔는지'는 정확하게 셀 수 있다

- BGE-M3 임베딩은 '의미'를 이해하지만, 'HBM' 같은 고유명사·약어는 놓치기도 합니다.
- BM25는 반대로 의미는 몰라도, 찾는 단어가 문서에 얼마나 자주·희귀하게 등장하는지를 계산해 점수를 매깁니다.
- 이번 단계에서는 이 BM25 검색 엔진을 준비합니다.

Step 1. BM25 — 문제 해결

이런 증상이 보이면	원인	해결 방법
결과가 하나도 없음	corpus와 query의 tokenize 방식이 다름	둘 다 <code>bm25s.tokenize()</code> 로 동일하게 처리했는지 확인
한국어 검색이 부정확함	띄어쓰기 기준이라 조사(은/는/이/가)가 섞임	오늘은 기본 토크나이저로 진행, 심화는 <code>konlpy</code> 참고(부록)

Dense가 실패하는 순간을 직접 목격하기

개념

"HBM"이라는 단어 자체보다, **'비슷한 분위기의 문장'**을 찾는 게 **Dense** 검색의 특징

- BGE-M3 임베딩(Dense)은 '의미가 비슷한 문장'을 찾는 데는 강하지만,
- 'HBM'처럼 고유한 약어 하나가 핵심인 질문에서는 엉뚱하게 '반도체 산업 개요' 같은 일반적인 문서를 가져오기도 합니다.
- 지금 두 검색을 나란히 실행해서 이 차이를 직접 확인해봅시다.

Dense 실패 케이스 직접 확인

❌ Dense 결과 (예상)

- 반도체 산업 개요
- AI 시장 동향
- 메모리 기술 소개
- 산업 전망 요약

BM25 결과 (예상)

- HBM 기술 상세 설명
- HBM 시장 점유율
- 차세대 HBM 로드맵

쿼리: "HBM" (고유명사·약어) — 직접 실행해서 두 결과를 비교해보세요

RRF — 두 검색 결과를 안전하게 합치는 법

개념

두 심사위원의 '점수'가 아니라 '등수'만 보고 합산한다

- Dense 점수(0~1)와 BM25 점수(0~수십)는 기준이 완전히 달라서 그냥 더하면 안 됩니다.
- 그래서 점수 대신 '몇 등이었는지(순위)'만 이용해서 안전하게 합산하는 방법이 RRF입니다.
- 두 검색 모두에서 상위권이었던 문서일수록 최종 점수가 높아집니다.

Step 3. RRF — 문제 해결

이런 증상이 보이면	원인	해결 방법
KeyError 또는 타입 에러	numpy int 와 python int 가 섞임	int(i) 로 명시적으로 형변환했는지 확인
Dense 후보와 BM25 후보가 안 겹침	정상입니다 — 그래서 Hybrid 가 필요한 것	겹치지 않아도 RRF 는 두 후보를 모두 고려하니 걱정 없음
특정 문서가 항상 1등 으로 나옴	두 검색 모두에서 상위였다는 뜻(정상)	이상하면 rrf_k(60) 나 k*3 범위를 조정해서 테스트

Reranker — 서류 전형 다음, 대면 면접

개념

빠르게 뽑은 후보를, 이번엔 꼼꼼하게 한 명씩 다시 읽어보고 채점한다

- RRF로 뽑은 후보들은 '빠르지만 대략적인' 방식으로 골라진 것들입니다.
- Reranker는 질문과 각 후보 문서를 '한 쌍씩 직접' 비교하며 훨씬 정밀하게 점수를 매깁니다.
- 전체 문서가 아니라 후보 20~50개만 다시 보기 때문에 시간이 오래 걸리지 않습니다.

Step 4. Reranker — 문제 해결

이런 증상이 보이면	원인	해결 방법
모델 다운로드가 오래 걸림	최초 실행 시 1회만 발생하는 다운로드	몇 분 정도 소요될 수 있음, 인터넷 연결만 확인하고 대기
메모리 부족(OOM) 에러	reranker도 모델이라 메모리를 사용함	use_fp16=True 유지, candidate_ids의 k값을 20 → 10으로 축소
pairs 형식 에러	리스트 안에 [질문,문서] 형태가 아님	pairs = [[query, c] for c in candidates] 형태인지 확인

세 단계 비교란 무엇을 확인하는 걸까?

개념

검색 방법을 한 단계씩 추가할 때마다, 결과가 실제로 좋아지는 걸 눈으로 확인하는 것

- 오늘 우리는 같은 질문(예: 'HBM 반도체')을 세 가지 다른 방식으로 검색해봤습니다.
 - ① Dense만 — Step 4까지 만든 BGE-M3 임베딩 검색 '단독' 결과
 - ② Hybrid(RRF) — 여기에 Step 1의 BM25(키워드 검색)까지 합친 결과
 - ③ +Reranker — Hybrid 결과를 Step 4에서 만든 Reranker로 한 번 더 정밀하게 재채점한 결과

세 단계 한눈에 정리

단계	이번 실습에서 실행한 함수	기대하는 변화
① Dense만	<code>index.search(q_vec, k=5)</code>	의미는 비슷하지만 'HBM'과 무관한 문서도 섞여 나옴
② Hybrid (RRF)	<code>hybrid_search_rrf(query, k=10)</code>	BM25 덕분에 'HBM' 키워드가 포함된 문서가 상위로 올라옴
③ +Reranker	<code>reranker.compute_score(pairs)</code>	후보들의 순서가 질문과의 관련도 기준으로 더 정밀해짐

이 세 함수를 같은 질문으로 각각 실행해서, 아래 표처럼 결과를 직접 비교합니다

Before / After — 세 단계 결과 비교 (예시)

순위	Dense만	Hybrid (RRF)	+Reranker
1위	반도체 산업 개요	HBM 기술 설명	HBM 기술 설명 ✓
2위	AI 시장 동향	HBM 시장 점유율	HBM 시장 점유율 ✓
3위	메모리 기술 소개	차세대 HBM 로드맵	차세대 HBM 로드맵 ✓

쿼리: "HBM 반도체" — 각 단계에서 순위가 실제로 어떻게 바뀌는지 직접 출력해서 확인해보세요

단계마다 무엇이 왜 좋아졌을까?

1 ① → ② **Dense만** → **Hybrid**

BM25가 'HBM'이라는 단어 자체를 직접 찾아주면서, 의미 검색이 놓친 키워드 매칭을 보완합니다

2 ② → ③ **Hybrid** → **+Reranker**

RRF로 뽑은 후보를 질문과 1:1로 비교해 재채점하면서 순위가 더 정밀해집니다

3 리포트에 쓸 내용

세 단계 각각의 실행 결과(캡처 또는 출력 텍스트)를 담고, '왜 개선됐는지' 한 줄씩 적으면 됩니다

이 세 단계 비교가 바로 제출 리포트에 들어갈 핵심 내용입니다

RAGAS — 감이 아니라 숫자로 확인하기

개념

"좋아진 것 같다"가 아니라 "**Faithfulness 0.6 → 0.9로 올랐다**"라고 말하기 위한 도구

- 지금까지는 결과를 눈으로 보고 '나아진 것 같다'고 판단했습니다.
- RAGAS는 질문·답변·검색된 문서만 넣으면 자동으로 여러 지표를 계산해줍니다.
- Faithfulness(근거에 충실한가), Answer Relevance(질문에 맞게 답했는가) 등을 점수로 확인

RAGAS로 결과 검증하기

RAG 전용 자동 평가 프레임워크

- **Faithfulness** — 답변이 검색된 문서 내용에 실제로 근거하는가?
- **Answer Relevance** — 답변이 질문의 의도에 맞게 대답하고 있는가?
- **Context Precision** — 검색된 문서 중 실제로 유용한 것의 비율은?

pip install ragas 로 설치, 질문-답변-검색결과만 있으면 바로 점수를 계산할 수 있습니다

Step 5. RAGAS — 문제 해결

이런 증상이 보이면	원인	해결 방법
OpenAI API 에러	RAGAS 내부적으로도 LLM을 호출해 채점함	OPENAI_API_KEY가 같은 터미널 환경에 설정돼 있는지 확인
점수가 낮게 나옴	정상적인 진단 결과일 수 있음	낮은 항목을 보고 청크 크기·검색 방식 중 무엇을 바꿀지 판단
실행이 느림	평가할 때마다 LLM을 호출하기 때문	질문 1~2개로 먼저 테스트 후 전체로 확장

실습 2 완료 체크 + 자유 실습

완료 체크리스트

- ☐ **BM25 인덱스 + Dense 실패 케이스 확인**
- ☐ **RRF 결합 함수 동작 확인 (Hybrid가 Dense보다 나은 것 확인)**
- ☐ **BGE-Reranker 적용 후 순위 변화 확인**
- ☐ **완전한 파이프라인으로 통합 질문 실행**

자유 실습: 다른 쿼리로 비교, RAGAS 점수 직접 계산, Parent-Child 구조 추가 실험

오늘 실습으로 직접 확인한 것들



임베딩 생성 시간

수백 개 청크에 수십 초
→ 10만개면 GPU가 필요한 이유



FAISS의 한계

메타데이터 저장 불가
→ 전용 서버(Qdrant)가 필요한 이유



Dense 실패 케이스

HBM 같은 고유명사 쿼리에서 직접 확인



RRF 결합 효과

키워드 관련 문서가 상위로 올라오는 것 확인



Reranker 순위 변화

같은 Top-5인데 순서가 달라지는 것 확인



RAGAS 검증

감으로만 판단하지 않고 숫자로 확인하는 습관

실습 제출물 안내

제출 대상 – 아래 3가지를 하나로 묶어도 무방 (캠퍼스_반_이름.zip으로 제출)

- ① 개발 코드 — 폴더 구조는 포함, 불필요한 폴더 제거
- ② 리포트 — 실습1 검색·필터 결과, 실습2 세 단계 비교 + **RAGAS** 검증 결과
+ **RAG** 설정 시 주요 고려 사항 정리 (개인 의견 중심)

자유 실습: 다른 쿼리로 비교, **RAGAS** 점수 직접 계산, **Parent-Child** 구조 추가 실험



수고하셨습니다.

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.